

EXERCÍCIO PRÁTICO

Manipulando objetos 3D com transformações e câmera virtual

Python + Pygame | Computação Gráfica / Realidade Estendida

Aula relacionada	Aula 6 — Manipulação de objetos no espaço tridimensional
Produto	Aplicação interativa que exibe um objeto-pai e um objeto-filho em uma cena 3D projetada em uma janela 2D
Duração sugerida	2 a 3 horas
Pré-requisitos	Python, estruturas de repetição, funções, vetores 3D e noções de projeção perspectiva

1. Contexto e objetivos

Em aplicações gráficas e ambientes virtuais, um objeto não é apenas uma malha: ele possui uma pose — posição, orientação e escala — em relação a um sistema de coordenadas. Alterar essa pose exige aplicar transformações e observar seus efeitos pela câmera virtual. Neste exercício, você construirá uma cena simples com um cubo-pai e um cubo-filho, implementando manualmente transformações lineares elementares e uma projeção perspectiva em Pygame.

- Representar um objeto 3D por vértices e arestas.
- Aplicar escala, rotação e translação em coordenadas locais.
- Compreender a composição e a ordem das transformações.
- Implementar uma relação hierárquica pai-filho.
- Manipular uma câmera virtual por deslocamento e observar a projeção.
- Relacionar a interação por teclado ao ciclo entrada → atualização → saída.

2. Enunciado

Desenvolva uma aplicação chamada “Manipulador 3D”. A janela deve apresentar dois cubos em perspectiva: um objeto-pai, em azul, e um objeto-filho, em amarelo. O usuário poderá selecionar cada objeto, movimentá-lo, rotacioná-lo e alterar sua escala. O objeto-filho deverá manter uma posição relativa ao pai, de modo que as transformações do pai também influenciem a pose final do filho. A câmera poderá ser deslocada para produzir diferentes pontos de vista da cena.

A implementação deve ser feita sem utilizar uma engine 3D pronta. O objetivo é tornar explícita a passagem por representação geométrica, transformação, câmera, projeção e rasterização de linhas.

3. Conceitos mobilizados

Modelo local	Vértices descritos em torno da origem do objeto, facilitando reutilização e transformação.
---------------------	--

Transformações	Escala, rotação em X/Y/Z e translação, preferencialmente aplicadas nessa ordem.
Composição	A ordem das operações altera o resultado; matrizes 4×4 em coordenadas homogêneas são a forma geral estudada em Computação Gráfica.
Hierarquia	O filho possui uma pose relativa ao pai; a pose mundial é obtida pela composição das poses.
Câmera	Posição do observador, direção de visão e distância influenciam a imagem resultante.
Projeção	Na perspectiva, pontos mais distantes aparecem menores: $x_s = f \cdot x/z$ e $y_s = f \cdot y/z$.

4. Preparação do ambiente

No terminal, crie uma pasta para a atividade, instale o Pygame e execute o programa:

```
python -m pip install pygame
python manipulador_3d.py
```

Em alguns sistemas, o comando de execução pode ser python3 manipulador_3d.py.

5. Código-base completo

Salve o código a seguir como manipulador_3d.py. Leia os comentários antes de executar e identifique as funções que correspondem a cada etapa do pipeline.

```
import math
import pygame

pygame.init()
LARGURA, ALTURA = 1000, 700
tela = pygame.display.set_mode((LARGURA, ALTURA))
pygame.display.set_caption("Manipulação de objetos 3D")
relógio = pygame.time.Clock()
fonte = pygame.font.SysFont("Arial", 18)

# Vértices de um cubo unitário, centrado na origem.
CUBO = [
    (-1,-1,-1), ( 1,-1,-1), ( 1, 1,-1), (-1, 1,-1),
    (-1,-1, 1), ( 1,-1, 1), ( 1, 1, 1), (-1, 1, 1)
]
ARESTAS = [(0,1),(1,2),(2,3),(3,0), (4,5),(5,6),(6,7),(7,4),
            (0,4),(1,5),(2,6),(3,7)]

def rotacionar(ponto, rx, ry, rz):
    x, y, z = ponto
    # Rotação em X
    c, s = math.cos(rx), math.sin(rx)
    y, z = y*c - z*s, y*s + z*c
    # Rotação em Y
    c, s = math.cos(ry), math.sin(ry)
    x, z = x*c + z*s, -x*s + z*c
    # Rotação em Z
    c, s = math.cos(rz), math.sin(rz)
```

```

    x, y = x*c - y*s, x*s + y*c
    return x, y, z

def transformar(ponto, pos, escala, rot):
    # Escala local, depois rotação e, por fim, translação.
    x, y, z = ponto
    x, y, z = x*escala[0], y*escala[1], z*escala[2]
    x, y, z = rotacionar((x,y,z), *rot)
    return x + pos[0], y + pos[1], z + pos[2]

def projetar(ponto, camera, foco=520):
    # Câmera simples: deslocamento no mundo e projeção perspectiva.
    x, y, z = ponto
    cx, cy, cz = camera
    x, y, z = x-cx, y-cy, z-cz
    z = max(z, 0.1)
    tela_x = LARGURA/2 + foco*x/z
    tela_y = ALTURA/2 - foco*y/z
    return int(tela_x), int(tela_y)

def desenhar_objeto(superficie, objeto, camera, cor):
    pontos = [projetar(transformar(v, objeto['pos'], objeto['escala'], objeto['rot']),
    camera) for v in CUBO]
    for a, b in ARESTAS:
        pygame.draw.line(superficie, cor, pontos[a], pontos[b], 3)

objeto = {'pos': [0, 0, 8], 'escala': [1.5, 1.5, 1.5], 'rot': [0, 0, 0]}
filho = {'pos': [0, 0, 2.8], 'escala': [0.55, 0.55, 0.55], 'rot': [0, 0, 0]}
camera = [0, 0, 0]
selecionado = 'pai'
rodando = True
while rodando:
    dt = relógio.tick(60) / 1000
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            rodando = False
        if evento.type == pygame.KEYDOWN:
            if evento.key == pygame.K_TAB:
                selecionado = 'filho' if selecionado == 'pai' else 'pai'
            if evento.key == pygame.K_r:
                objeto['pos'], objeto['rot'], objeto['escala'] = [0,0,8], [0,0,0],
[1.5,1.5,1.5]
                filho['pos'], filho['rot'], filho['escala'] = [0,0,2.8], [0,0,0],
[0.55,0.55,0.55]

    teclas = pygame.key.get_pressed()
    alvo = objeto if selecionado == 'pai' else filho
    velocidade = 3 * dt
    # Translação do objeto selecionado no plano XZ.
    if teclas[pygame.K_LEFT]: alvo['pos'][0] -= velocidade
    if teclas[pygame.K_RIGHT]: alvo['pos'][0] += velocidade
    if teclas[pygame.K_UP]: alvo['pos'][2] -= velocidade
    if teclas[pygame.K_DOWN]: alvo['pos'][2] += velocidade
    # Rotação em torno dos eixos X e Y.
    if teclas[pygame.K_a]: alvo['rot'][1] -= 1.8 * dt
    if teclas[pygame.K_d]: alvo['rot'][1] += 1.8 * dt
    if teclas[pygame.K_w]: alvo['rot'][0] -= 1.8 * dt
    if teclas[pygame.K_s]: alvo['rot'][0] += 1.8 * dt
    # Escala uniforme.
    if teclas[pygame.K_q]: alvo['escala'] = [max(0.2, v-1.2*dt) for v in alvo['escala']]
    if teclas[pygame.K_e]: alvo['escala'] = [v+1.2*dt for v in alvo['escala']]
    # Manipulação da câmera: I/K sobe/desce, J/L desloca lateralmente.

```

```

    if teclas[pygame.K_j]: camera[0] -= 3*dt
    if teclas[pygame.K_l]: camera[0] += 3*dt
    if teclas[pygame.K_i]: camera[1] += 3*dt
    if teclas[pygame.K_k]: camera[1] -= 3*dt

    tela.fill((18, 25, 38))
    # O filho é desenhado em coordenadas do pai: relação hierárquica simples.
    desenhar_objeto(tela, objeto, camera, (60, 190, 255))
    filho_mundo = dict(filho)
    filho_mundo['pos'] = [objeto['pos'][i] + filho['pos'][i] for i in range(3)]
    filho_mundo['rot'] = [objeto['rot'][i] + filho['rot'][i] for i in range(3)]
    desenhar_objeto(tela, filho_mundo, camera, (255, 190, 70))

    texto = f"Selecionado: {selecionado} | TAB alterna | Setas move | A/D e W/S gira |
Q/E escala"
    texto2 = "J/L e I/K movem a câmera | R restaura"
    tela.blit(fonte.render(texto, True, (235,235,235)), (20, 20))
    tela.blit(fonte.render(texto2, True, (190,205,220)), (20, 48))
    pygame.display.flip()
pygame.quit()

```

6. Controles da aplicação

TAB	Alterna a seleção entre o objeto-pai e o objeto-filho.
Setas	Movem o objeto selecionado no plano XZ.
A / D	Rotacionam o objeto selecionado em torno do eixo Y.
W / S	Rotacionam o objeto selecionado em torno do eixo X.
Q / E	Reduzem ou aumentam a escala uniforme.
J / L	Movem a câmera para a esquerda ou direita.
I / K	Movem a câmera para cima ou para baixo.
R	Restauram as poses iniciais dos objetos.

7. Roteiro de desenvolvimento

1. Execute o código e identifique os vértices e as arestas do cubo.
2. Altere apenas a translação do pai. Observe o deslocamento do filho e explique a relação hierárquica.
3. Altere a ordem entre escala, rotação e translação na função transformar. Registre uma situação em que a troca produz resultado diferente.
4. Adicione uma rotação em torno do eixo Z e crie um controle para ela.
5. Implemente uma grade no plano XZ para auxiliar a percepção de profundidade.
6. Inclua uma indicação visual do objeto selecionado, como mudança de cor ou marcador no centro.
7. Teste posições próximas e distantes da câmera. Observe a variação aparente do tamanho.

8. Questões para discussão

- Por que os vértices são definidos em coordenadas locais?
- Por que a ordem escala → rotação → translação foi adotada no código?
- O que acontece com o filho quando o pai é rotacionado? O código implementa uma hierarquia completa ou uma aproximação?
- Qual problema ocorre quando z é zero ou negativo na projeção perspectiva?
- Como a projeção ortográfica alteraria o comportamento visual da cena?
- Quais parâmetros adicionais seriam necessários para uma câmera definida por posição, ponto focal e vetor view up?
- Como esse exercício poderia ser adaptado para uma interação em Realidade Virtual ou Aumentada?

9. Extensões opcionais

- Adicionar uma pirâmide ou outro poliedro.
- Implementar matrizes 4×4 e coordenadas homogêneas em vez de calcular as operações separadamente.
- Criar uma câmera orbital com azimuth, elevação e distância ao redor do objeto.
- Adicionar recorte simples para não desenhar arestas atrás do plano de recorte.
- Implementar seleção por clique usando uma aproximação de traço de raio.
- Exibir os valores atuais de posição, rotação e escala na tela.
- Salvar e carregar poses em um arquivo JSON.

10. Entrega esperada

Entregue o arquivo manipulador_3d.py e um relatório curto, de uma a duas páginas, contendo: (a) descrição da solução; (b) imagem da aplicação em execução; (c) explicação da hierarquia pai-filho; (d) exemplo que demonstre a importância da ordem das transformações; e (e) uma proposta de extensão para um ambiente XR.

11. Critérios de avaliação sugeridos

Representação geométrica — 20%	Cubo definido por vértices/arestas e desenho coerente na janela.
Transformações — 25%	Translação, rotação e escala funcionando e aplicadas de forma compreensível.
Hierarquia — 20%	Objeto-filho com pose relativa e resposta adequada às transformações do pai.
Câmera e projeção — 15%	Manipulação da câmera e projeção perspectiva sem falhas básicas.
Interação e organização — 10%	Controles claros, aplicação responsiva e código organizado.
Análise e criatividade — 10%	Respostas às questões, registro de testes e extensão opcional.

12. Síntese

A atividade transforma os conceitos da Aula 6 em uma experiência observável: uma malha é descrita em coordenadas locais, recebe transformações, participa de uma hierarquia, é vista por uma câmera e finalmente é projetada e desenhada na tela. Essa sequência é uma versão reduzida do pipeline usado em aplicações gráficas interativas e prepara o estudo de dispositivos de entrada, interação e traço de raio nas próximas aulas.